

Think of the difference between **Liana** and **xta** as the difference between a **specialized research prototype** and an **industry-standard legacy format**.

While both describe the same thing—**Timed Automata**—they were designed for different purposes and different engines.

1. The Core Philosophy

The biggest difference isn't just the syntax, but the "math" they are built to support.

- **Liana (for TARZAN):** Designed specifically for **Region-based** analysis. It is "strict" and focused on precision. It loves **punctual guards** ($x == 5$) because regions can pinpoint the exact moment a clock hits an integer without losing accuracy. 
- **XTA (for UPPAAL):** Designed for **Zone-based** analysis. Zones are much faster for most systems but can be "blunt." XTA is built to support massive systems with complex C-like logic, often at the cost of the granular precision that regions provide.

2. Syntax Style: Tuples vs. Templates

The way you "write" the automaton is fundamentally different.

Liana: The "Mathematical Tuple" Style

Liana is very "tuple-heavy." To define a transition, you provide a list of elements in a specific order:

```
(source, action, clock_guard, bool_expr, resets, target)
```

It reads like a formal mathematical definition. It's compact, but you have to count the commas to make sure you're in the right spot.

XTA: The "C-like / Template" Style

XTA uses a system of **Templates**. You define a general "template" of how a process works, then **instantiate** it with specific names later. [🔗 +1](#)

```
process P1() { ... }
```

```
system P1, P2;
```

XTA's transitions are defined by labels (like `guard`, `assign`, `sync`) rather than a fixed-position tuple. It feels much more like writing code in C or Java.

3. Key Feature Comparison

Feature	Liana	XTA (UPPAAL)
Punctual Guards	Native Strength. Handles <code>x == 5</code> perfectly using Regions.	Supportive but Slow. Zones can struggle with these in large models.
Symmetry	Explicit. Has a <code>symm</code> rule to group identical processes.	Implicit. Uses "Scalar Sets" to achieve a similar effect.
Logic	Simple Boolean and Arithmetic.	Advanced "C-like" functions, arrays, and complex structs.
Structure	Single file, direct automaton definition.	Multi-part: Declarations, Templates, and System Instantiation.

4. Why your Parser matters

Because you are translating from **Liana** to **XTA**, you are essentially "unpacking" Liana's dense tuples and "re-packaging" them into XTA's labeled format.

- **Your Challenge:** When Liana says `[x, y]` for resets, XTA needs you to write `x := 0, y := 0`.
 - **Your Challenge:** When Liana uses `T / F`, XTA expects `true / false`.
 - **Your Advantage:** By moving a Liana model to XTA, you can use UPPAAL's GUI to **visualize** the automaton that was previously just a text file in TARZAN.
-

In summary:

Liana is like a high-precision scientific instrument: hard to read, but perfect for specific timing proofs. **XTA** is like a Swiss Army knife: it does everything, is used by everyone, but isn't always the most precise tool for every single timing "cut."